머신러닝

7주차. 결정트리 하이퍼 파라미터

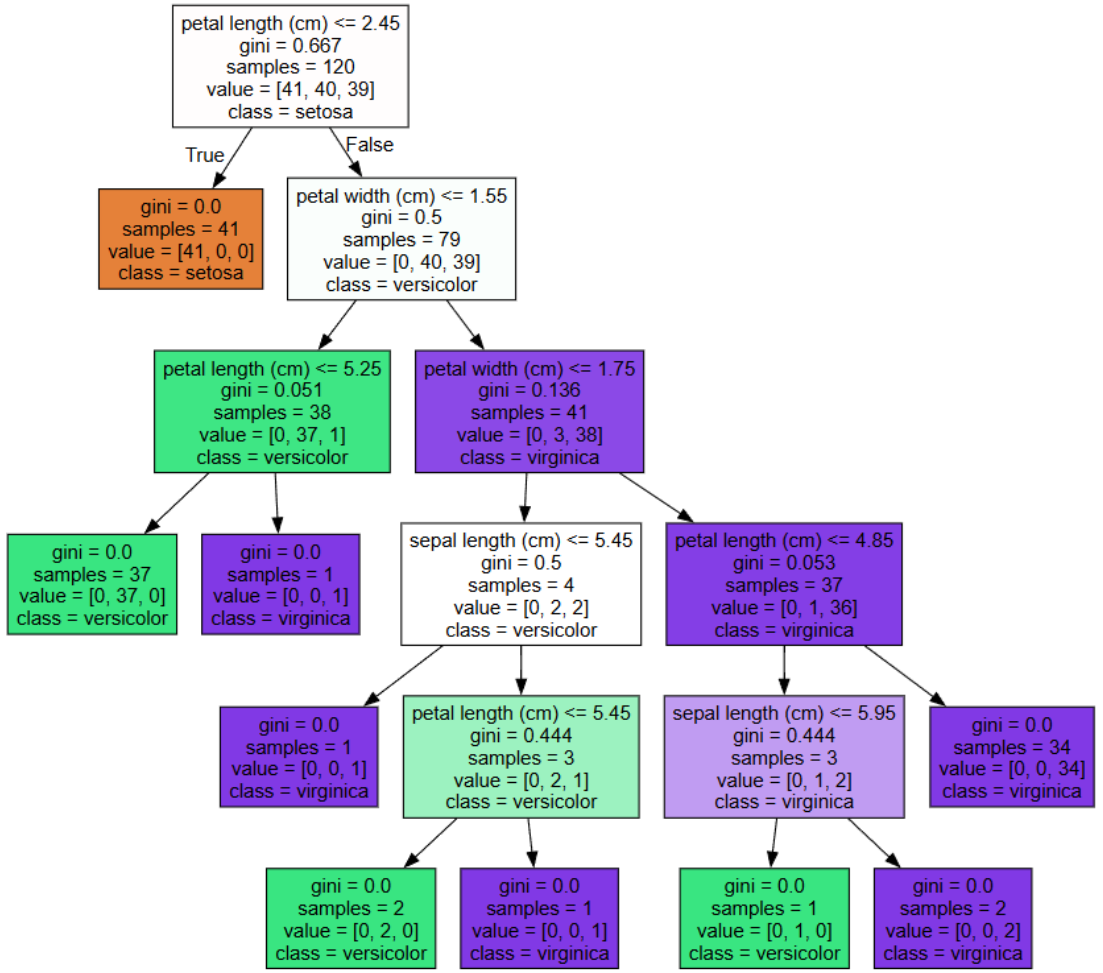
학습목표

- **결정트리 하이퍼 파라미터**
- **결정 트리 피처 중요도**
- **결정 트리 과적합**
  - 샘플 데이터 생성
- **결정 트리 실습**
  - 위스콘신 유방암 데이터셋

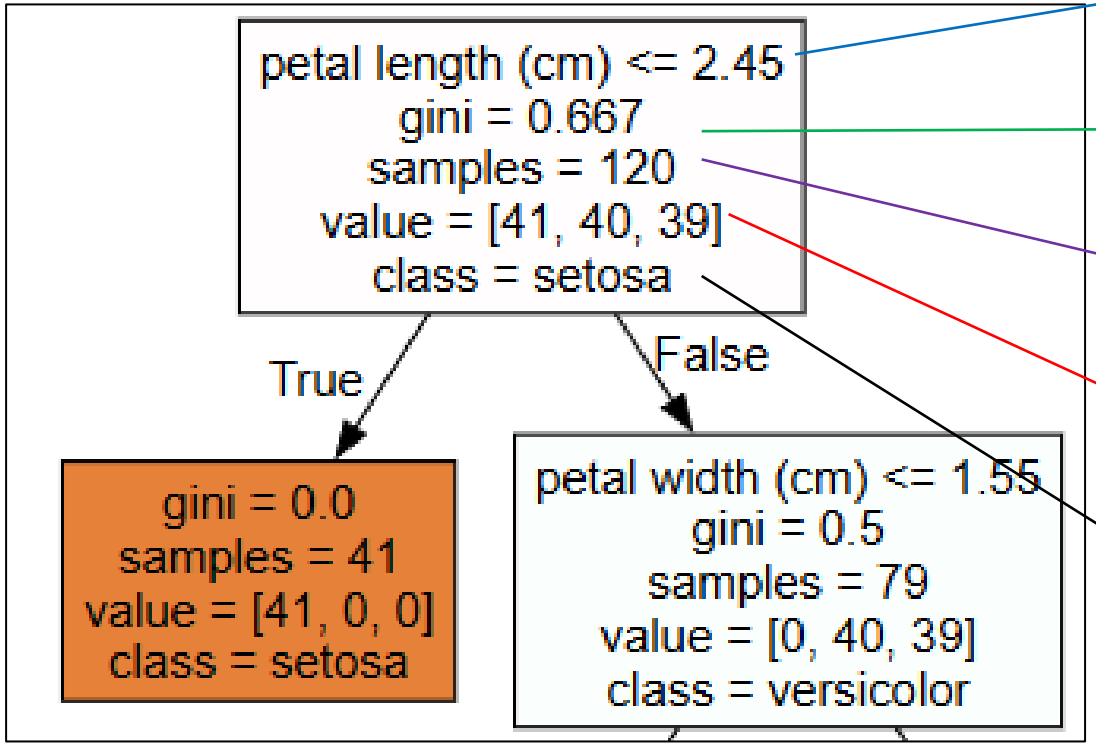
결정 트리 하이퍼 라마미터

- 결정 트리 학습 결과 및 시각화 결과

```
DecisionTreeClassifier(
    *,
    criterion='gini',
    splitter='best',
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    min_weight_fraction_leaf=0.0,
    max_features=None,
    random_state=None,
    max_leaf_nodes=None,
    min_impurity_decrease=0.0,
    class_weight=None,
    ccp_alpha=0.0,
)
```



결정 트리 하이퍼 라마미터(계속)

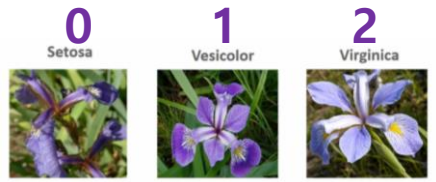


분할 조건(petal length (cm) <= 2.45
- 자식 노드를 만들기 위한 규칙 조건
- 이 조건이 없으면 리프노드

Gini
- 지니계수

samples
- 현재 노드의 데이터 수

value
- 클래스 값 기반의 데이터 수



class
- value 리스트에서 가장 많은 건수 조건에 의해 분류되는 클래스

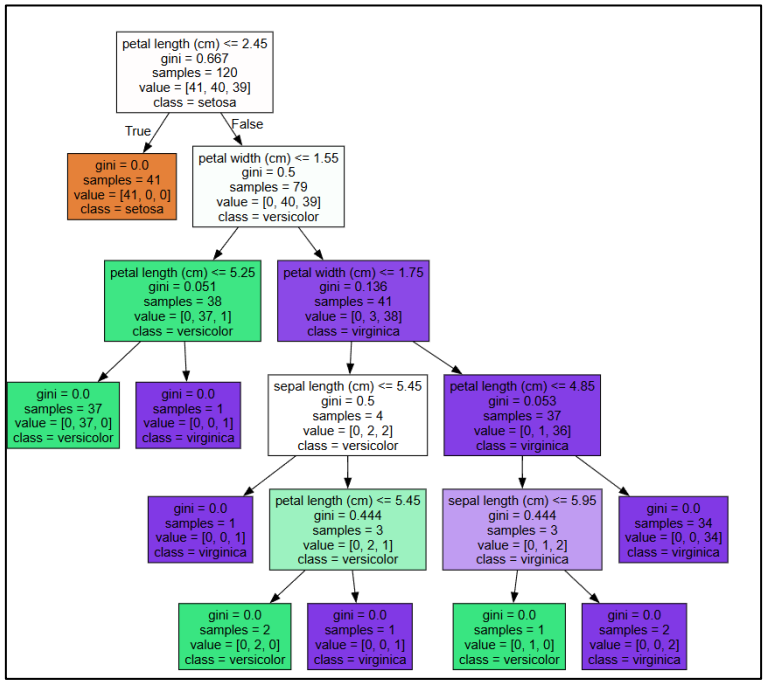
결정 트리 주요 하이퍼 파라미터

파라미터 명	설명
max_depth	• 트리의 최대 깊이 • 기본값(None)으로 설정되면 완벽하게 클래스 결정 값이 될 때 까지 깊이를 계속 키우며 분할하거나 노드가 가지는 데이터 개수가 **min_samples_split 보다 작아질 때 까지 계속 깊이 증가** • **깊이가 깊어지면 min_sample_split 설정대로 최대 분할하여 과적합**할 수 있으므로 적절한 값으로 제어 필요
max_features	• 최적의 분할을 위해 고려한 **최대 특징 개수**. 기본값(None)은 모든 특징을 사용해 분할 수행 • int형으로 지정하면 대상 특징의 개수, float형으로 지정하면 전체 피처중 대상 특징의 비율 • 'sqrt'/'auto'는 전체 특징 중 제곱급수 만큼 선정: $\sqrt{\text{전체 피처}}$개수 만큼 선정 • log는 전체 특징중 log2(전체 특징개수) 선정
min_samples_split 기본값(2)	• 노드를 분할하기 위한 최소한의 샘플데이터 수: 과적합을 제어하는데 사용 • 작게 설정할수록 분할되는 노드가 많아져서 과적합 가능성 증가
min_samples_leaf	• 분할이 될 경우 왼쪽과 오른쪽의 가지(브랜치)**노드에서 가져가야 할 최소한의 샘플 데이터 수** • **큰 값으로 설정될 수록**, 분할될 경우 좌우 가지 노드에서 가져야 할 최소한의 샘플데이터 수 조건을 만족시키기 어려워져 노드 분할을 상대적으로 덜 수행
max_leaf_nodes	• 말단 노드(Leaf)의 최대 개수

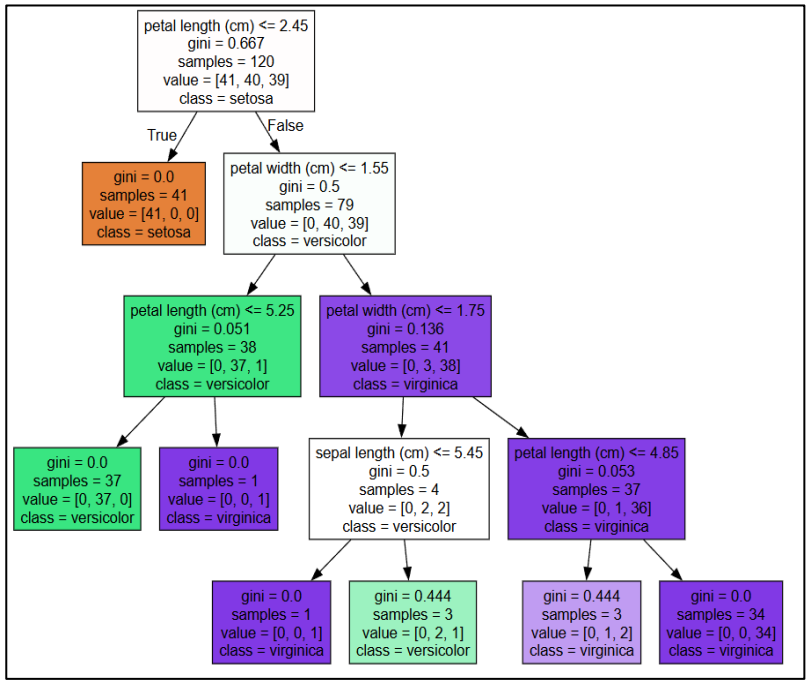
결정 트리 주요 하이퍼 파라미터(계속)

- **max_depth 값에 따른 결정 트리 구조**

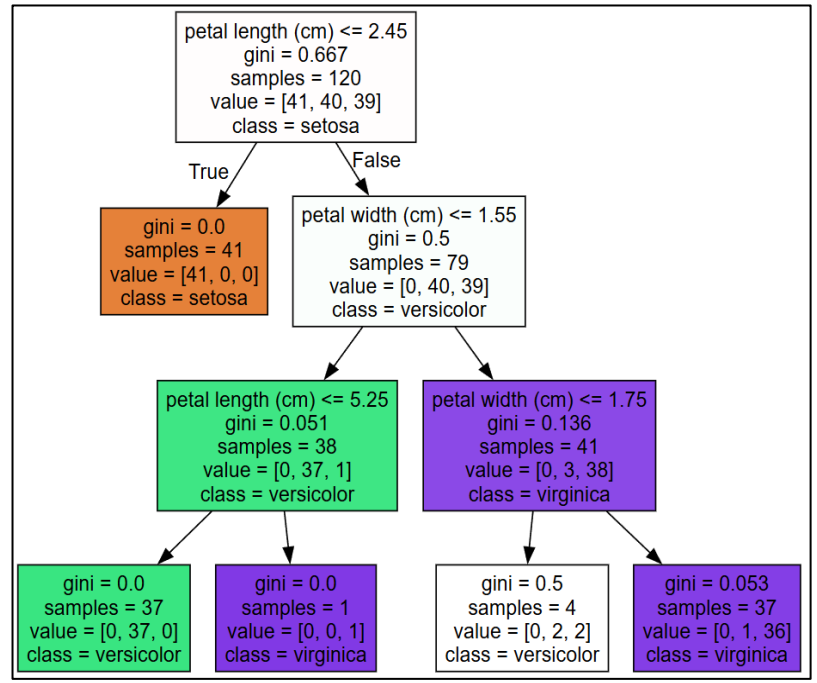
max_depth = None



max_depth = 4



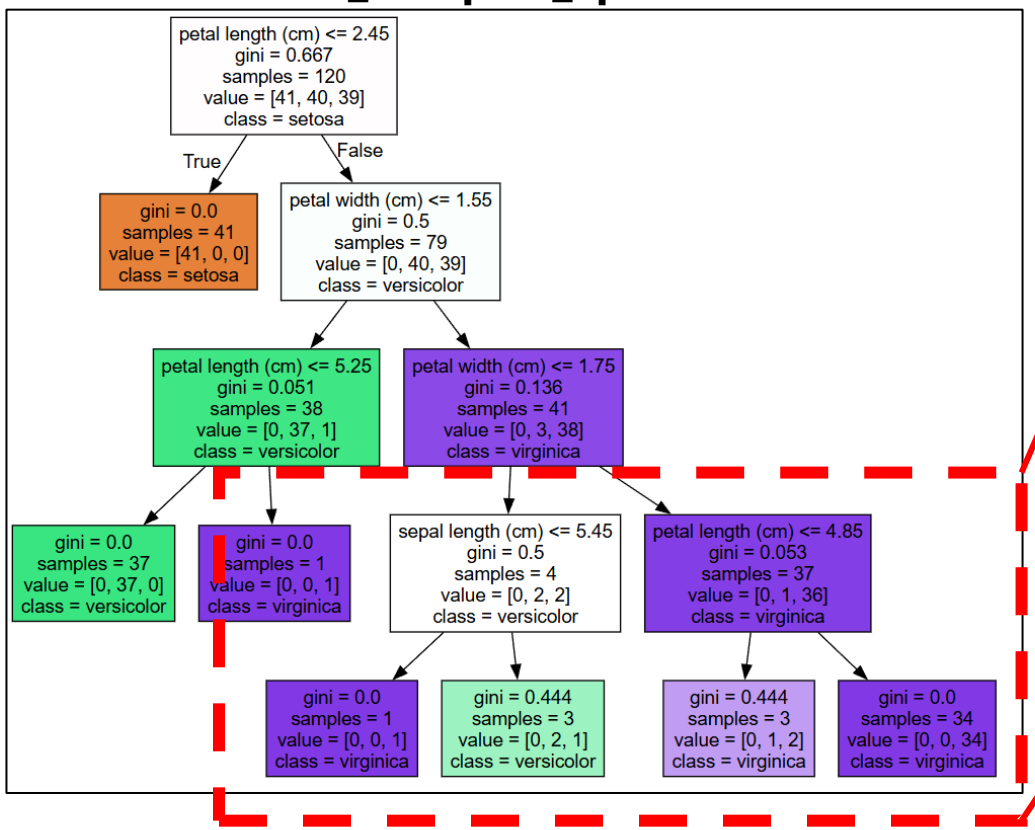
max_depth = 3

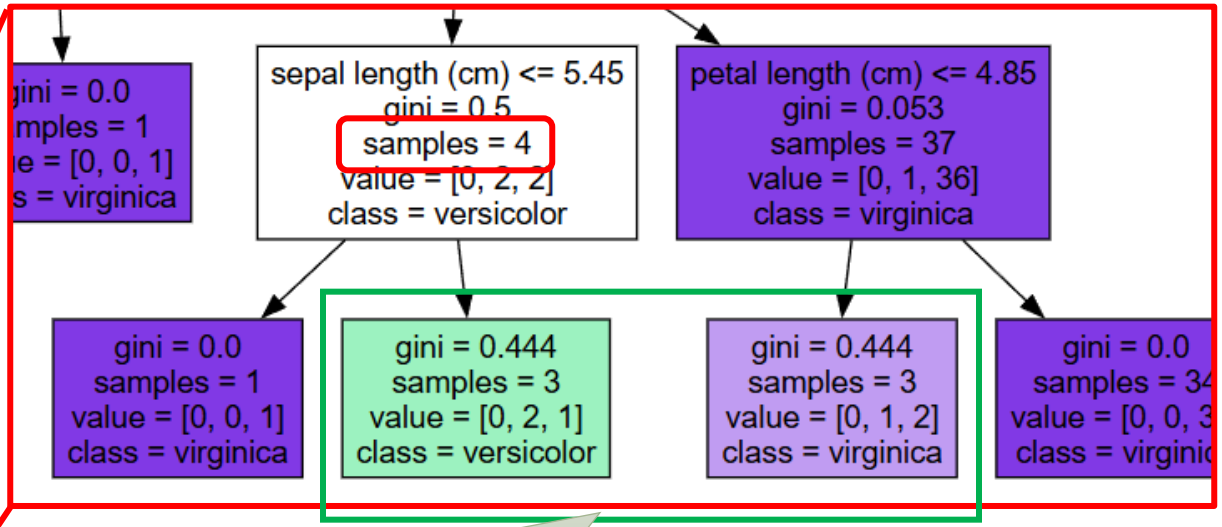


결정 트리 주요 하이퍼 파라미터(계속)

- **min_samples_split에 따른 결정 트리 구조**

min_samples_split = 4





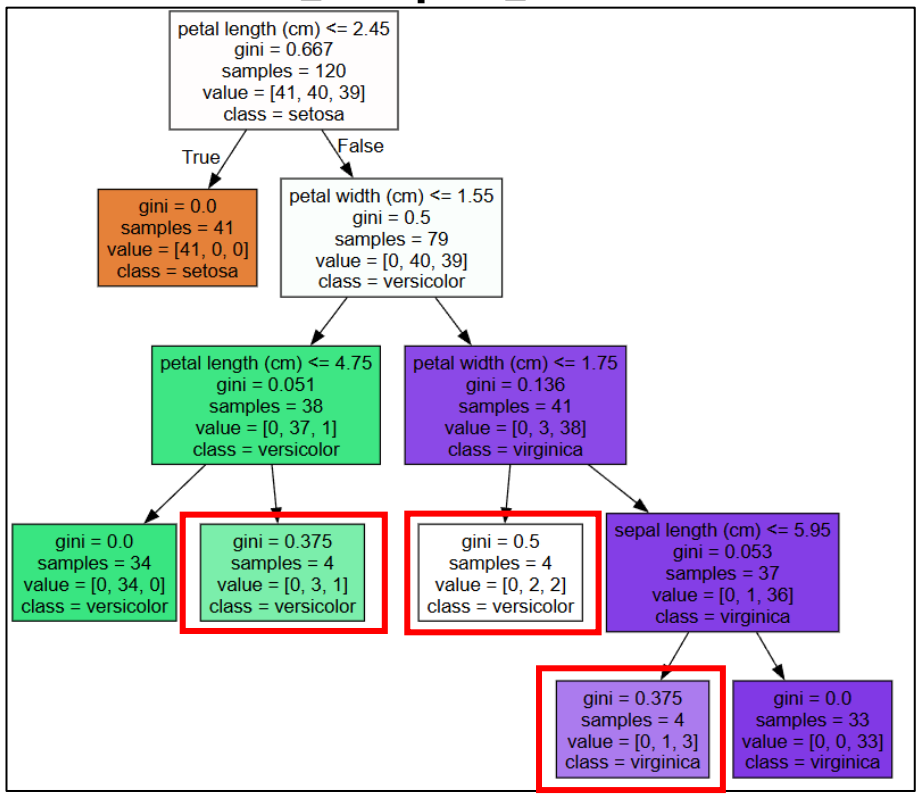
min_samples_split = 4로 설정

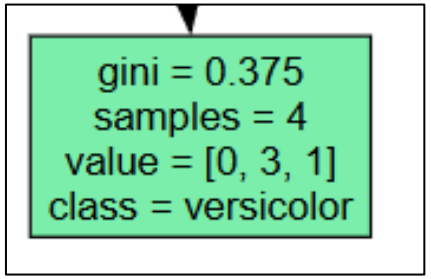
samples 의 값이 3으로 떨어지면 해당 노드에 다른 클래스가 섞여 있어도 더 이상 분할하지 않음

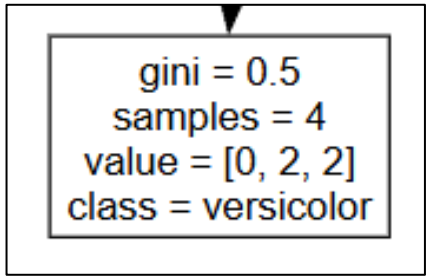
결정 트리 주요 하이퍼 파라미터(계속)

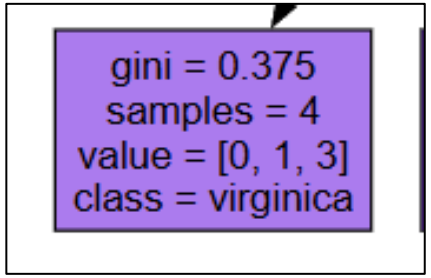
- **min_samples_leaf에 따른 결정 트리 구조**

min_samples_leaf = 4









min_samples_leaf = 4로 설정

분할이 되어서 데이터의 수가 4이상을 만족하는지 확인하고 분할을 수행

결정 트리 피처 중요도

- **결정 트리 피처 중요도**
  - 노드를 분할할 때, 해당 피처가 클래스를 나누는데 얼마나 영향을 미쳤는지를 표기하는 척도
  - 학습 시 분할에 사용된 피처가 상대적으로 얼마나 중요도를 갖는지에 대한 정도
  - 클래스 속성명

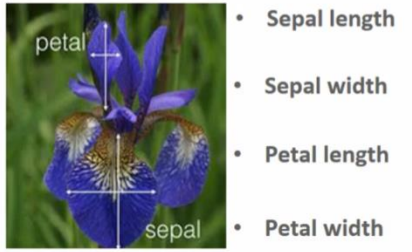
    결정트리객체.feature_importances_

```
import numpy as np

# feature importance 추출
print("Feature importances:\n{0}".format(np.round(dt_clf.feature_importances_ , 3)))
print()

# feature importance 매핑
for name, value in zip(iris_data.feature_names, dt_clf.feature_importances_):
    print("{0} : {1:.3f}".format(name, value))
```

iris_feature_name = 특징 이름




```
Feature importances:
[0.025 0.    0.555 0.42 ]

sepal length (cm) : 0.025
sepal width (cm) : 0.000
petal length (cm) : 0.555
petal width (cm) : 0.420
```

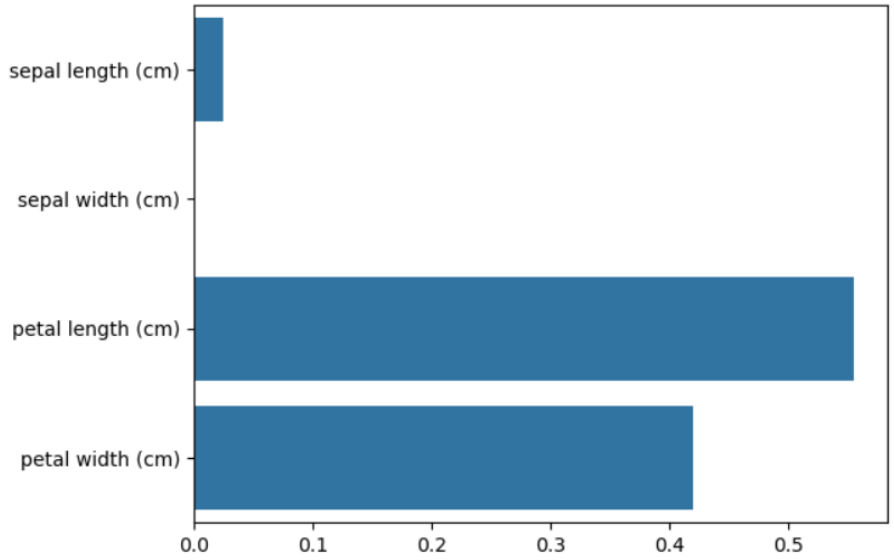
결정 트리 피처 중요도(계속)

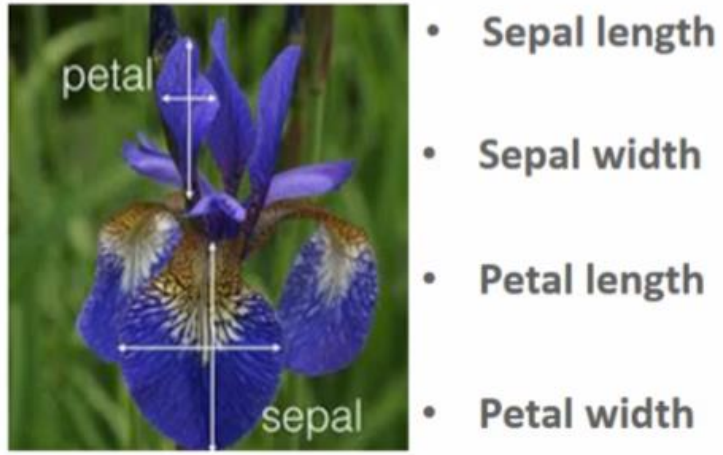
- 결정 트리 피처 중요도 시각화

```
import seaborn as sns

sns.barplot(x = dt_clf.feature_importances_, y = iris_data.feature_names)
```

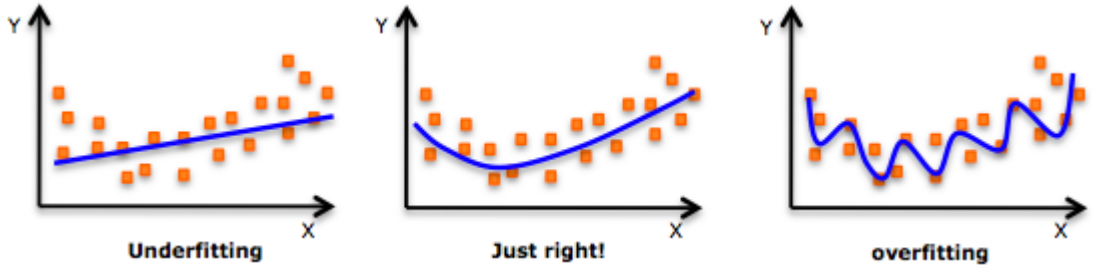
<Axes: >

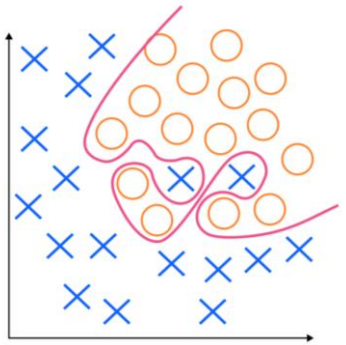




결정 트리 과적합

- **과적합(Overfitting)**
  - 모델이 훈련 데이터의 패턴뿐 아니라 그 안의 노이즈(Noise)나 **우연한 특성**까지 학습
  - 훈련 데이터과 검증 데이터에는 높은 정확도를 보이나 새로운 데이터에는 오히려 성능이 떨어지는 상태
  - 머신러닝 모델이 학습 데이터에는 매우 잘 맞지만, 새로운 데이터에는 일반화 성능이 떨어지는 현상





결정 트리 과적합(계속)

- **과적합의 발생 원인**
  - 모델 복잡도 과다: 파라미터 수가 너무 많은 복잡한 모델
    ex): 깊은 신경망, 고차 다항 회귀 등)
  - 데이터 부족: 학습 데이터가 적어 일반적인 패턴을 포착하기 어렵고, 특이한 샘플에 지나치게 적응함
  - 노이즈 포함: 데이터 자체에 불필요한 잡음이 많을 경우 이를 '의미 있는 패턴'으로 잘못 학습
  - 정규화 부족: 규제(Regularization)가 없으면 모델이 자유롭게 파라미터를 조정해 버림

- **과적합 해결 방법**
  - 더 복잡한 모델 사용
  - 학습 시간 늘리기
  - 더 풍부한 feature 추가
  - 학습률 조정 및 적절한 하이퍼 파라미터 탐색

결정 트리 과적합
- 샘플 데이터 생성

```
from sklearn.datasets import make_classification
```

- **make_classification()**
  - 분류 문제를 위한 가상 데이터 셋을 생성
  - 특성(feature), 클래스(class), 샘플(sample) 수 등을 설정
  - 머신러닝 모델의 학습, 실험 또는 시각화 테스트용으로 사용

  - **호출 파라미터**

    n_samples: 샘플 데이터의 개수(필수: default = 100)
    n_features: 전체 특징 개수(필수: default = 20)
    n_informative: 독립변수 중 종속 변수와 상관 관계가 있는 성분의 수
    n_redundant: 다른 특성의 선형 결합으로 만들어지는 특성 개수
    n_classes: 종속 변수의 클래스 개수
    n_clusters_per_class: 각 클래스가 몇 개의 클러스터로 구성 될 지의 수

  - **반환 값(X, y)**
    - X : [n_samples, n_features] 크기의 ndarray(독립변수)
    - y : [n_samples] 크기의 ndarray(종속변수)

```
make_classification(
    n_samples=100,
    n_features=20,
    *,
    n_informative=2,
    n_redundant=2,
    n_repeated=0,
    n_classes=2,
    n_clusters_per_class=2,
    weights=None,
    flip_y=0.01,
    class_sep=1.0,
    hypercube=True,
    shift=0.0,
    scale=1.0,
    shuffle=True,
    random_state=None,
)
```

결정 트리 과적합
- 샘플 데이터 생성(계속)

- **샘플 데이터 생성**
  - feature의 개수 : 2
  - class의 개수 : 3

```
from sklearn.datasets import make_classification

# 2차원 시각화를 위해서 feature는 2개, target 클래스는 3개의 샘플 데이터 생성
X_features, y_labels =  make_classification(n_features = 2,
                                            n_redundant = 0,
                                            n_informative = 2,
                                            n_classes = 3,
                                            n_clusters_per_class = 1,
                                            random_state = 0)

print(X_features.shape)
print(y_labels.shape)
print()

print("생성된 X_features 데이터: \n", X_features[:3])
print()
print("생성된 y_labels 데이터: \n", y_labels[:10])
```

```
(100, 2)
(100,)

생성된 X_features 데이터:
 [[ 0.12041909 -1.68941375]
 [ 1.09665605  1.79445113]
 [ 0.88426577  1.43370121]]

생성된 y_labels 데이터:
 [0 1 1 1 2 2 1 0 2 2]
```

결정 트리 과적합
- 샘플 데이터 생성(계속)

- **샘플 데이터 삼전도 코드**

```
import matplotlib.pyplot as plt
import matplotlib
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline

# 한글 폰트 깨짐 방지 코드
matplotlib.rcParams['font.family'] ='Malgun Gothic'
matplotlib.rcParams['axes.unicode_minus'] =False

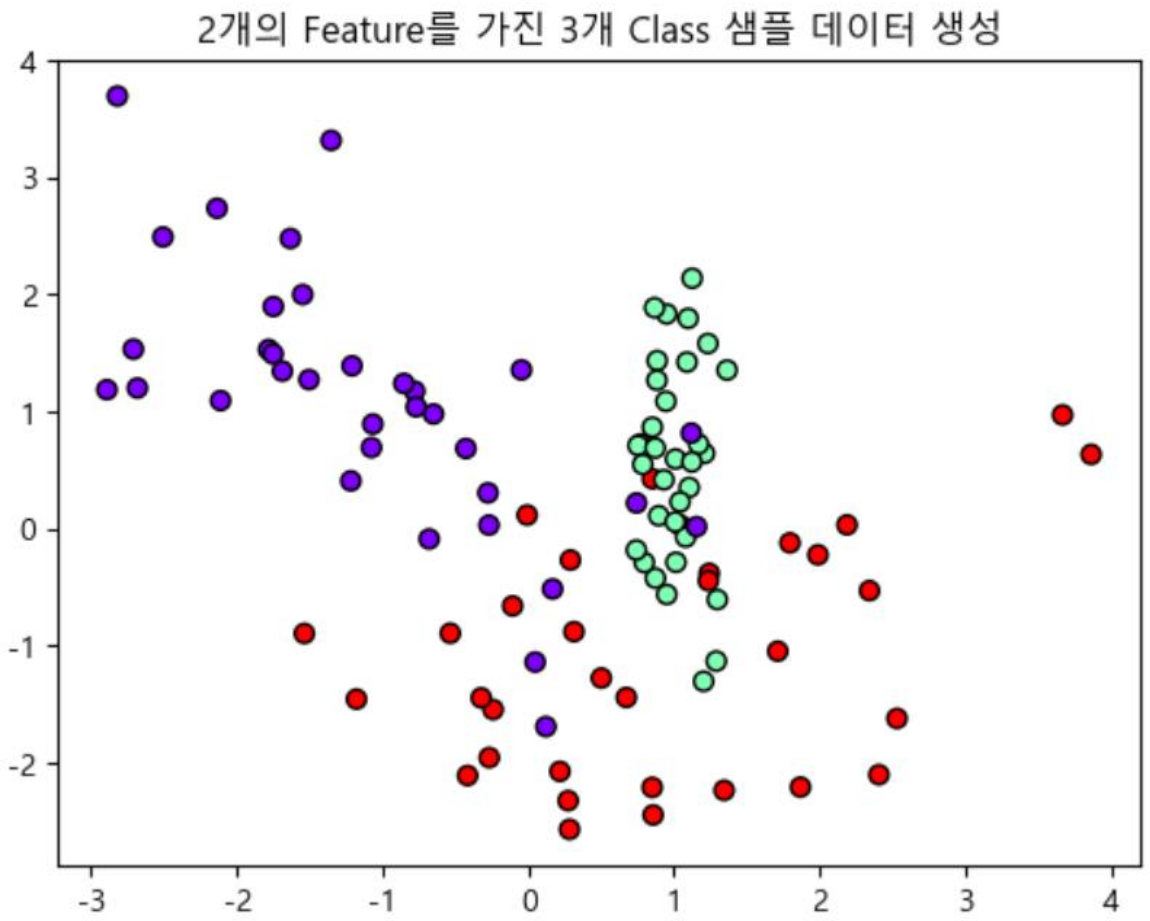
plt.title("2개의 Feature를 가진 3개 Class 샘플 데이터 생성")

# 산점도(Scatter)로 생성된 데이터의 분포를 시각화 /  각 클래스 값은 다른 색깔로 표시
plt.scatter(X_features[:, 0], X_features[:, 1],
            marker = 'o', c = y_labels,
            s = 40, cmap='rainbow', edgecolor = 'k')

plt.show()
```

결정 트리 과적합
- 샘플 데이터 생성(계속)

- 샘플 데이터의 삼전도(Scatter) 결과



결정 트리 과적합
- 샘플 데이터 생성(계속)

- **Decision Boundary 시각화 함수**

```
import numpy as np

# Classifier의 Decision Boundary를 시각화 하는 함수
def visualize_boundary(model, X, y):
    fig, ax = plt.subplots()

    # 학습 데이터 scatter plot으로 나타내기
    ax.scatter(X[:, 0], X[:, 1], c=y, s=25, cmap='rainbow', edgecolor='k',
               clim=(y.min(), y.max()), zorder=3)
    ax.axis('tight')
    ax.axis('off')
    xlim_start , xlim_end = ax.get_xlim()
    ylim_start , ylim_end = ax.get_ylim()

    # 호출 파라미터로 들어온 training 데이타로 model 학습 .
    model.fit(X, y)
    # meshgrid 형태인 모든 좌표값으로 예측 수행.
    xx, yy = np.meshgrid(np.linspace(xlim_start,xlim_end, num=200),np.linspace(ylim_start,ylim_end, num=200))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # contourf() 를 이용하여 class boundary 를 visualization 수행.
    n_classes = len(np.unique(y))
    contours = ax.contourf(xx, yy, Z, alpha=0.3,
                           levels=np.arange(n_classes + 1) - 0.5,
                           cmap='rainbow', clim=(y.min(), y.max()),
                           zorder=1)
```

결정 트리 과적합
- 샘플 데이터 과적합

- 기본 학습 모델에 Decision Boundary 시각화

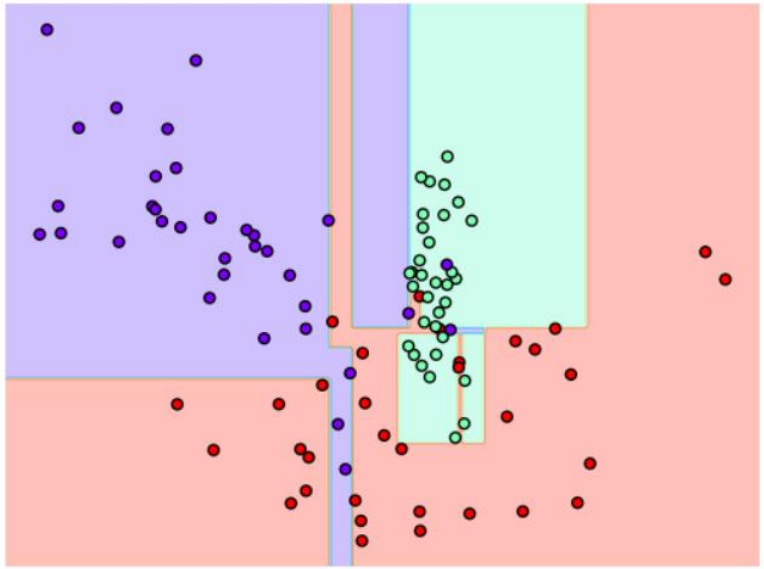
```
from sklearn.tree import DecisionTreeClassifier

dt_clf = DecisionTreeClassifier(random_state = 156).fit(X_features, y_labels)
visualize_boundary(dt_clf, X_features, y_labels)
```

학습된
모델 객체

생성된
F_features

생성된
y_labels

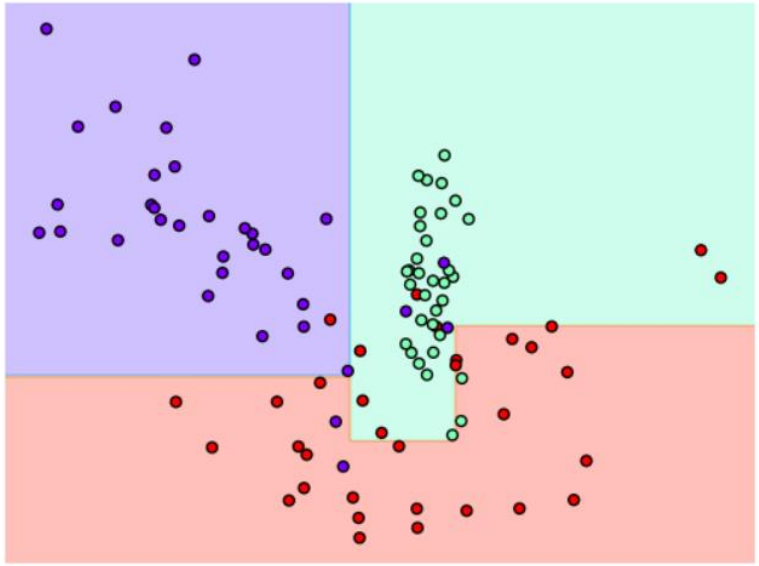


결정 트리 과적합
- 샘플 데이터 과적합(계속)

- 하이퍼 파라미터값이 변경된 모델에 Decision Boundary 시각화

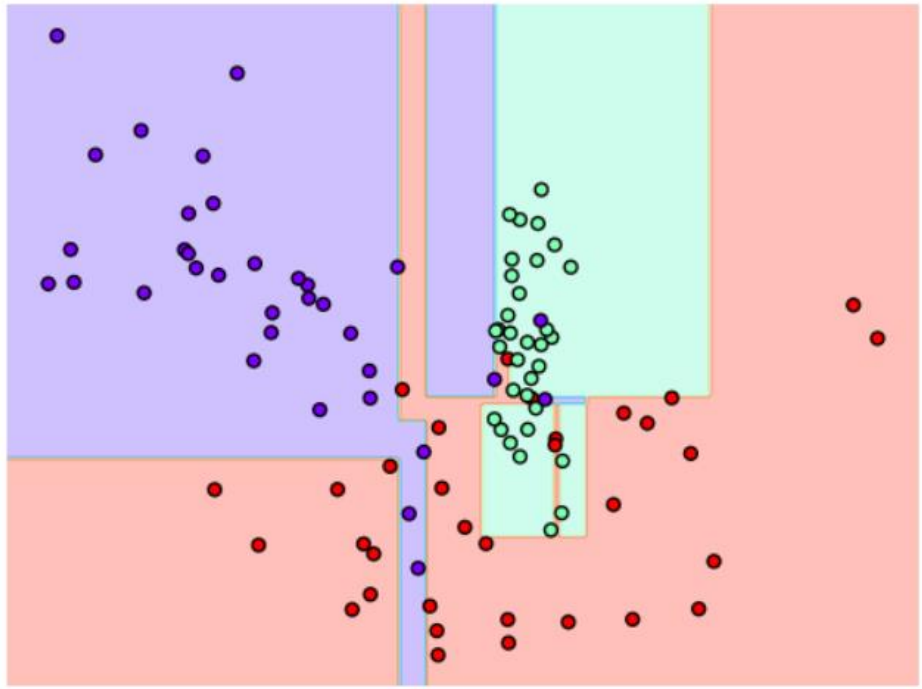
```
dt_clf = DecisionTreeClassifier(random_state = 156,
                                min_samples_leaf = 6).fit(X_features, y_labels)
visualize_boundary(dt_clf, X_features, y_labels)
```

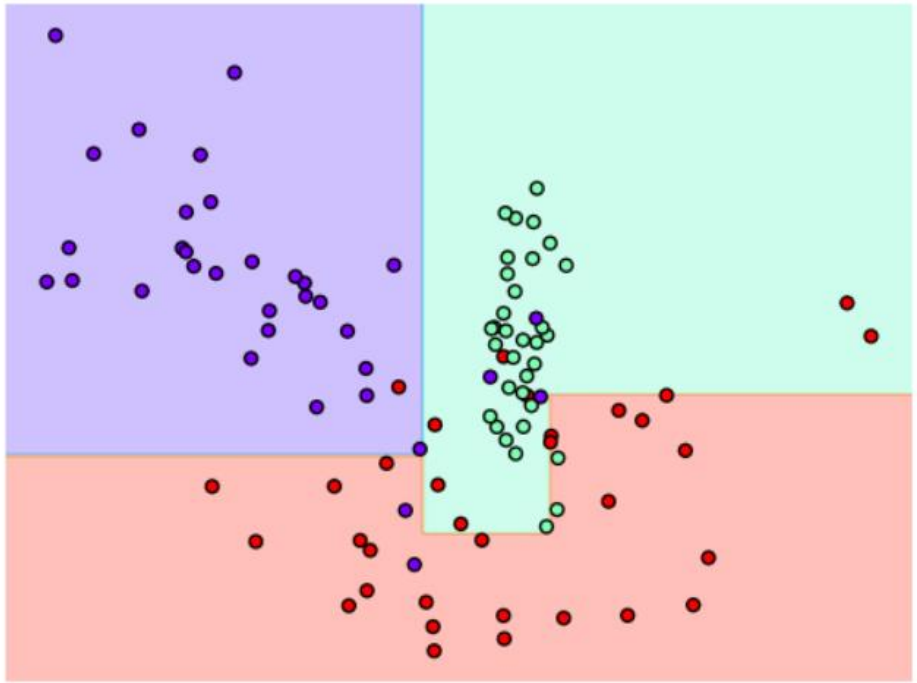
분류될 노드의 데이터 개수가
6이상일 때 분류



결정 트리 과적합
- 샘플 데이터 과적합

- **Decision Boundary 비교**



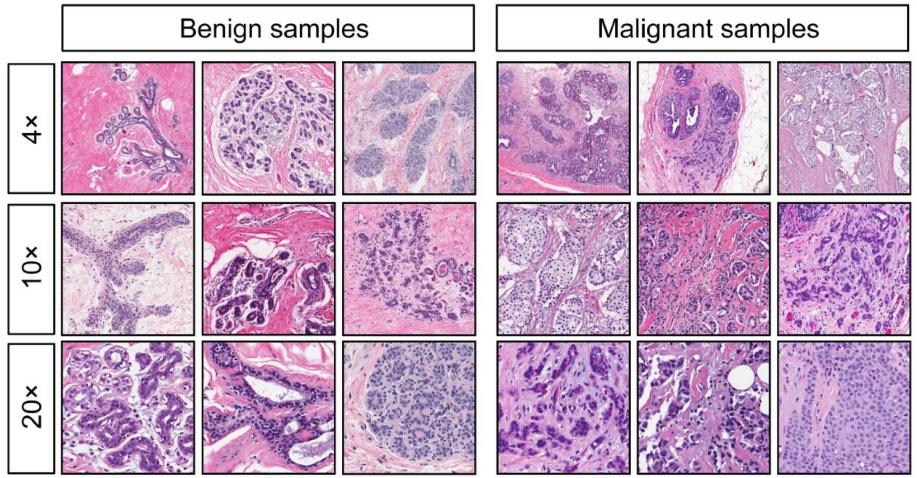


결정 트리 실습
- 유방암 데이터

- **위스콘신 유방암 데이터셋(Wisconsin Breast Cancer Dataset)**
  - scikit-learn에서 제공하는 대표적인 이진 분류(classification) 데이터셋
  - 유방 종양이 양성(Benign)인지 악성(Malignant)인지를 예측하는 데 사용

```
from sklearn.datasets import load_breast_cancer
```

항목	내용
데이터	Breast Cancer Wisconsin (Diagnostic) Dataset
출처	UCI Machine Learning Repository
제공자	Dr. William H. Wolberg (University of Wisconsin Hospitals)
샘플 수	569개 (악성 212개, 양성 357개)
특징(Features)	30개의 연속형(real-valued) 변수
타깃(Target)	0 = 악성(Malignant), 1 = 양성(Benign)
문제유형	이진 분류(Binary Classification)



결정 트리 실습
- 유방암 데이터 (계속)

- **데이터 구조**
  - 유방 조직 세포의 디지털 이미지로 부터 추출된 특징들
  - 각 특성은 세포 핵(nucleus)의 형태적 속성을 나타냄
    mean(평균), error(standard error: 표준오차), worst(최대값)의 통계치 포함

특성 그룹	예시 변수	설명
Radius (반지름)	`mean radius`	중심으로부터 주변까지 거리의 평균
Texture (조직)	`mean texture`	그레이 레벨(회색조)의 표준편차
Perimeter (둘레)	`mean perimeter`	셀의 둘레
Area (면적)	`mean area`	셀의 면적
Smoothness (매끄러움)	`mean smoothness`	반지름의 국소적 변동성
Compactness (조밀도)	`mean compactness`	둘레²/면적 – 1.0
Concavity (오목성)	`mean concavity`	오목한 부분의 심각도
Concave points (오목 분포)	`mean concave points`	오목한 부분의 개수
Symmetry (대칭성)	`mean symmetry`	대칭성 정도
Fractal Dimension (프랙탈 차원)	`mean fractal dimension`	경계의 복잡도 측정

결정 트리 실습
- 유방암 데이터의 결정트리 학습 - 데이터 로드

- 데이터 로드 및 GridSearchCV를 위한 하이퍼 파라미터 설정

```python
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
import pandas as pd
import numpy as np

# 1) 데이터 로드 & 확인
bc = load_breast_cancer()

bc_data = bc.data

bc_df = pd.DataFrame(data = bc_data, columns = bc.feature_names)
bc_df['label'] = bc.target

bc_df.head()

X_train, X_test, y_train, y_test = train_test_split(bc.data, bc.target, test_size=0.25, random_state=42, stratify = bc.target)

# 2) GridSearchCV를 위한 파라미터 설정
param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [None, 3, 4, 5, 6, 8],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "class_weight": [None, "balanced"]
}
```

bc(breast_cancer) 객체

bc.data = 특징 데이터(문제)
bc.target = 타겟 값(숫자_정답)
bc_feature_name = 특징 이름
bc.target_name = 레이블 이름

결정트리 하이퍼 파라미터(딕셔너리)
- criterion
- max_depth
- min_samples_split
- min_samples_leaf
- class_weight

결정 트리 실습
- 유방암 데이터의 결정트리 학습 - 데이터 로드(계속)

- **유방함 데이터 구조**
  - bc_df.head()

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...

worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	label
17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890	0
23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	0
25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	0
26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	0
16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	0

결정 트리 실습
- 유방암 데이터의 결정트리 학습 - 최적화

- **GridSearchCV를 통한 성능 최적화 학습**

```python
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, roc_auc_score

# 3) GridSearchCV (ROC_AUC 평가값으로 최적화)
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    scoring="roc_auc",
    cv=5,
    n_jobs=-1,
    return_train_score=True
)
grid.fit(X_train, y_train)

print("[Best Params]", grid.best_params_)
print("[CV Best ROC-AUC]:", grid.best_score_)

best_clf = grid.best_estimator_
```

GridSearchCV 의 입력값

```
GridSearchCV(
    estimator,  ── 기계학습 모델
    param_grid, ── 찾고자 하는 파라미터, 딕셔너리 형식
    *,
    scoring=None, ──예측평가지표
    n_jobs=None,
    refit=True, ── 최적의 파라미터로 모델로 생성
    cv=None,    ──교차검증 폴드수
    verbose=0,
    pre_dispatch='2*n_jobs',
    error_score=nan,
    return_train_score=False,
)
```

학습 후 GridSearchCV()의 반환값의 학습 결과의 속성값

속성	값
.cv_results_	하이퍼파라미터와 교차검증 결과 값(딕셔너리)
.best_params_	최적의 하이퍼 파라미터
.best_score_	최고의 정확도
.best_estimator_	최적의 하이퍼 파라미터로 모델 생성

결정 트리 실습
- 유방암 데이터의 결정트리 학습 - 평가

- **GridSearchCV를 통한 성능 최적화 학습 결과 및 ROC-AUC 평가**

```python
# 4) 최적화 학습후 하이퍼 파라미터 값, 평가 점수 확인

print("최적화 결과")
print("[Best Params]", grid.best_params_)
print("[CV Best ROC-AUC]:", grid.best_score_)
print()

# 5) 테스트셋 평가: ROC-AUC와 정확도 모두 보기
proba = best_clf.predict_proba(X_test)[:, 1]      # 1값으로 분류 될 확률
y_pred = best_clf.predict(X_test)                 # 예측 결과

print("테스트 셋 평가")
print("[Test ROC-AUC]:", roc_auc_score(y_test, proba))
print("[Test Accuracy]:", accuracy_score(y_test, y_pred))
```

학습 후 GridSearchCV()의 반환값의 학습 결과의 속성값

속성	값
.cv_results_	하이퍼파라미터와 교차검증 결과 값(딕셔너리)
.best_params_	최적의 하이퍼 파라미터
.best_score_	최고의 정확도
.best_estimator_	최적의 하이퍼 파라미터로 모델 생성

```
최적화 결과
[Best Params] {'class_weight': 'balanced', 'criterion': 'entropy', 'max_depth': 3, 'min_samples_leaf': 4, 'min_samples_split': 10}
[CV Best ROC-AUC]: 0.966295669337932

테스트 셋 평가
[Test ROC-AUC]: 0.9306079664570232
[Test Accuracy]: 0.916083916083916
```

결정 트리 실습
- 유방암 데이터의 결정트리 학습 - 트리 피처 중요도

- **트리 피처 중요도**

```
# 최적화 후 중요 피처 shape
print(best_clf.feature_importances_.shape)
print(bc.feature_names.shape)

# pandas Series로 데이터 변환, index: 피처 이름
data_pd = pd.Series(data = best_clf.feature_importances_ , index = bc.feature_names )

print(data_pd)
```

특성 그룹	예시 변수	설명
Radius (반지름)	radius_mean	중심으로부터 주변까지 거리의 평균
Texture (조직)	texture_mean	그레이 레벨(회색조)의 표준편차
Perimeter (둘레)	perimeter_mean	셀의 둘레
Area (면적)	area_mean	셀의 면적
Smoothness (매끄러움)	smoothness_mean	반지름의 국소적 변동성
Compactness (조밀도)	compactness_mean	둘레²/면적 − 1.0
Concavity (오목성)	concavity_mean	오목한 부분의 심각도
Concave points	concave_points_mean	오목한 부분의 개수
Symmetry (대칭성)	symmetry_mean	대칭성 정도
Fractal Dimension (프랙탈 차원)	fractal_dimension_mean	경계의 복잡도 측정

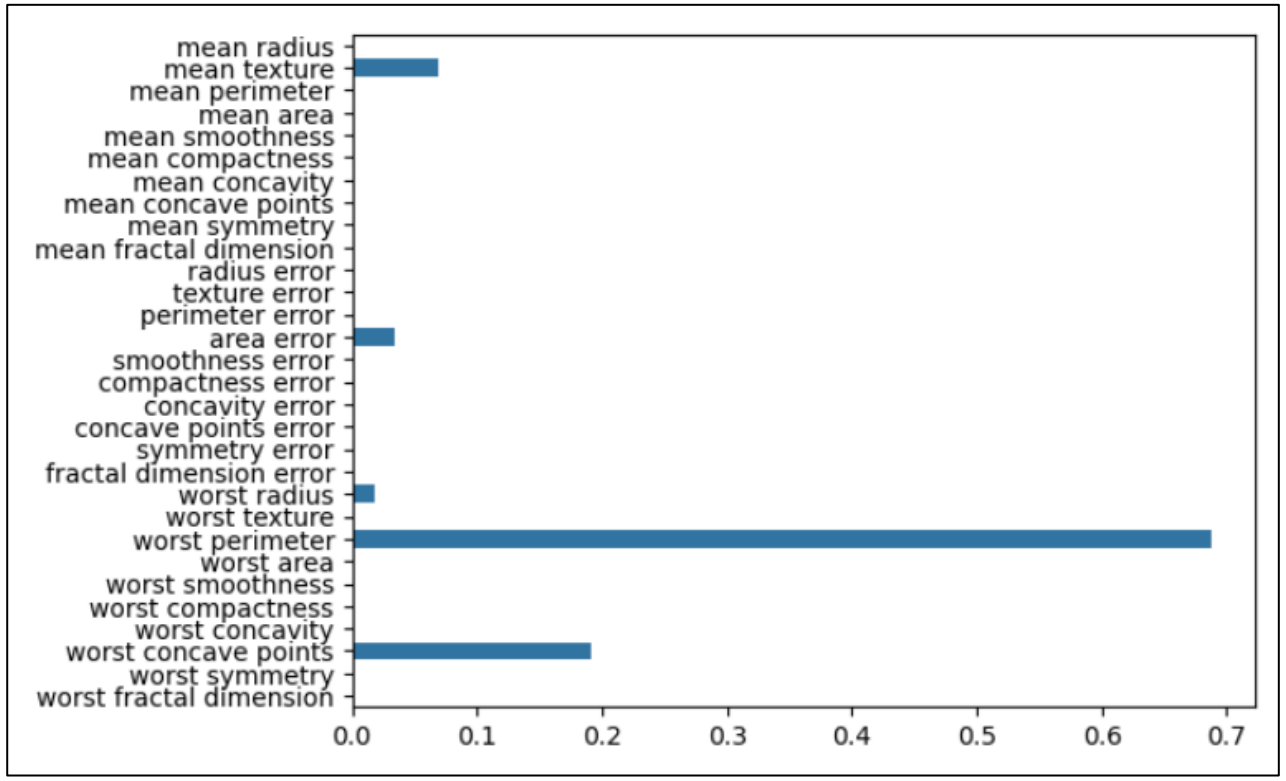
```
(30,)
(30,)
mean radius                0.000000
mean texture               0.068879
mean perimeter             0.000000
mean area                  0.000000
mean smoothness            0.000000
mean compactness           0.000000
mean concavity             0.000000
mean concave points        0.000000
mean symmetry              0.000000
mean fractal dimension     0.000000
radius error               0.000000
texture error              0.000000
perimeter error            0.000000
area error                 0.033863
smoothness error           0.000000
compactness error          0.000000
concavity error            0.000000
concave points error       0.000000
symmetry error             0.000000
fractal dimension error    0.000000
worst radius               0.018539
worst texture              0.000000
worst perimeter            0.687605
worst area                 0.000000
worst smoothness           0.000000
worst compactness          0.000000
worst concavity            0.000000
worst concave points       0.191114
worst symmetry             0.000000
worst fractal dimension    0.000000
dtype: float64
```

결정 트리 실습
- 유방암 데이터의 결정트리 학습 - 트리 피처 중요도(계속)

- 트리 피처 중요도(시각화)

```
import seaborn as sns

sns.barplot(x = best_clf.feature_importances_ , y = bc.feature_names)
```



결정 트리 실습
- 유방암 데이터의 결정트리 학습 – ROC-AUC Curve

- **ROC-AUC Curve**

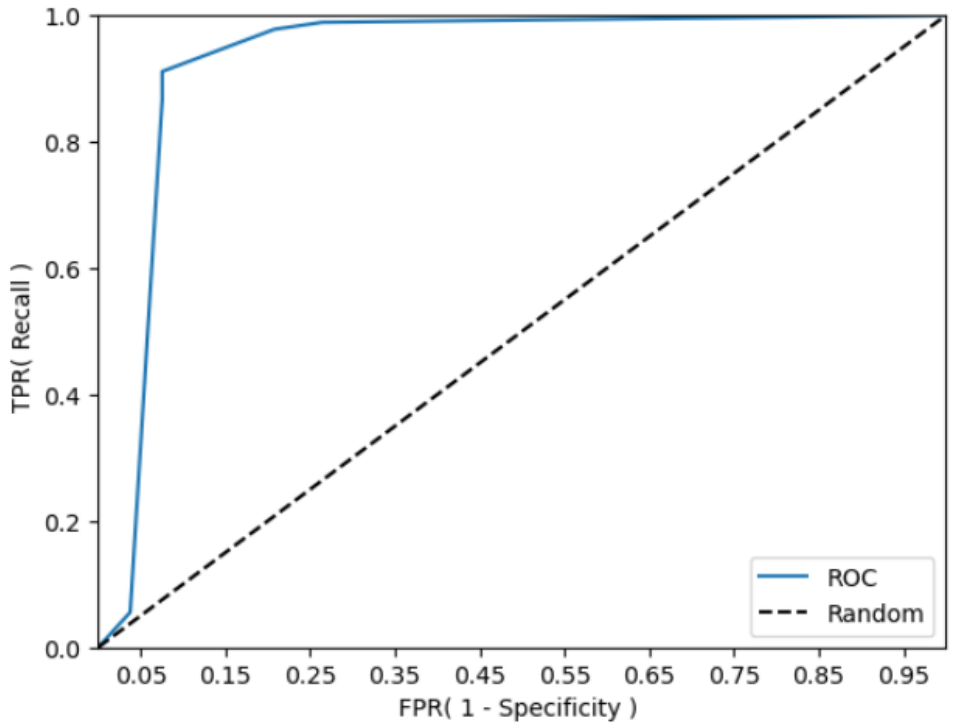
```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt
%matplotlib inline

def roc_curve_plot(y_test, pred_proba_c1):
    # 임곗값에 따른 FPR, TPR 값을 반환 받음
    fprs, tprs, thresholds = roc_curve(y_test, pred_proba_c1)

    # ROC Curve를 plot 곡선으로 그림
    plt.plot(fprs, tprs, label = "ROC" )
    #가운데 대각선 직선으로 그림
    plt.plot([0, 1], [0, 1], 'k--', label = "Random")

    # FPR X축의 Scale을 0,1 단위로 설정, X, Y 축명 설정
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1), 2))
    plt.xlim(0, 1)
    plt.ylim(0, 1)
    plt.xlabel('FPR( 1 - Specificity )')
    plt.ylabel('TPR( Recall ) ' )
    plt.legend()
    plt.show()

roc_curve_plot(y_test, best_clf.predict_proba(X_test)[:, -1])
```



결정 트리 실습
- 유방암 데이터의 결정트리 학습 – 시각화

- **Graphviz 시각화**

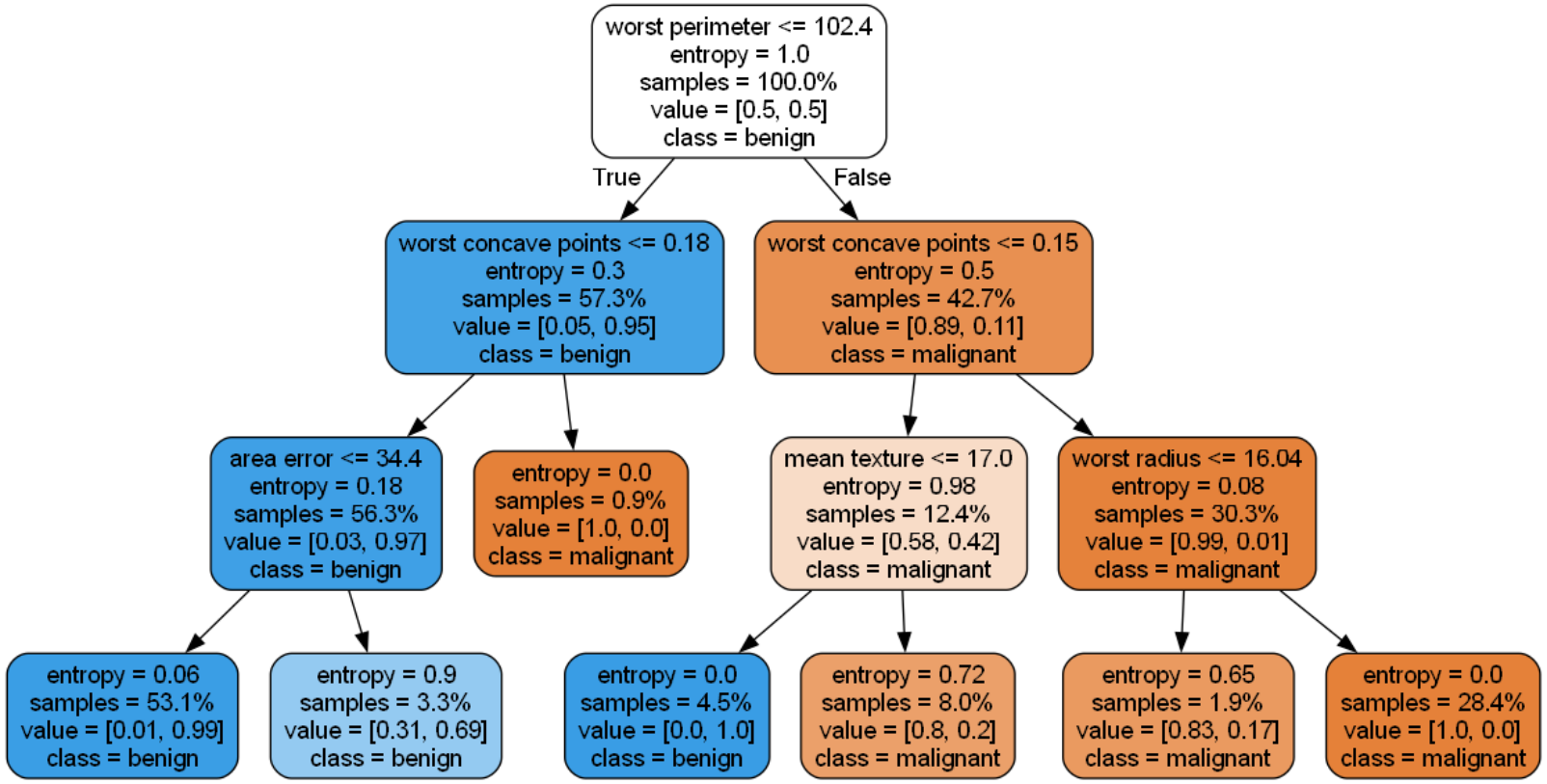
```
from sklearn.tree import export_graphviz
import graphviz

dot_data = export_graphviz(
    best_clf,
    out_file=None,
    feature_names=load_breast_cancer().feature_names,
    class_names=load_breast_cancer().target_names,
    filled=True,            # 색상 채우기 (불순도 표현)
    rounded=True,           # 모서리 둥글게
    proportion=True,         # 각 노드의 비율 표시
    precision=2              # 소수점 자리수
)

graph = graphviz.Source(dot_data)
graph.render("breast_cancer_tree", format="png", cleanup=True)  # PNG로 저장
graph
```

결정 트리 실습
- 유방암 데이터의 결정트리 학습 – 시각화(계속)

- **Graphviz 시각화**





강남대학교

KANGNAM UNIVERSITY